# Drawing 2D Computer Graphics

**W.A.G.Nilnuwani**
**Department of Information Technology,**
**Faculty of Information Technology**

# Introduction

- A **pixel** is the smallest display unit on a screen. Each image on the screen is formed by many pixels arranged in rows and columns and **Resolution** refers to the number of pixels used to display an image.

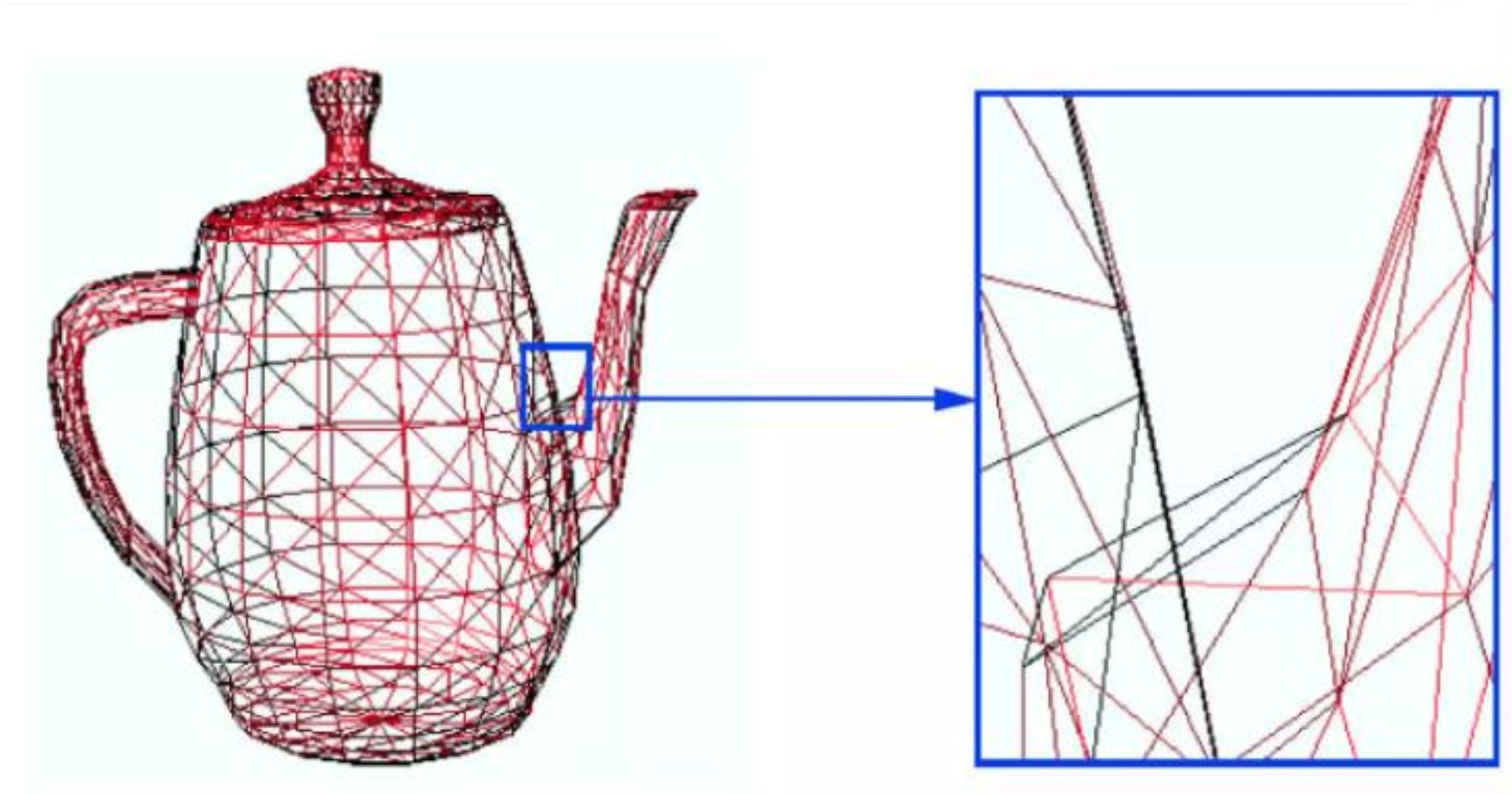
**Coordinate System**

- To draw anything on the screen, we need positions.
- In computer graphics,
  - origin is usually at the **top-left corner**
  - x increases to the right
  - y increases downward
- **Example**
  - Point (100, 50) means,
  - move 100 pixels to the right
  - move 50 pixels down

# Introduction contd..

- Graphics software and hardware provide subroutines to describe a scene using basic geometric structures called **output primitives**.
- Output primitives can be combined to create more complex structures.
- The simplest output primitives are,
  - Point (pixel)
  - line
  - circle
  - ellipse
  - polygon
  - curve

# Object Model

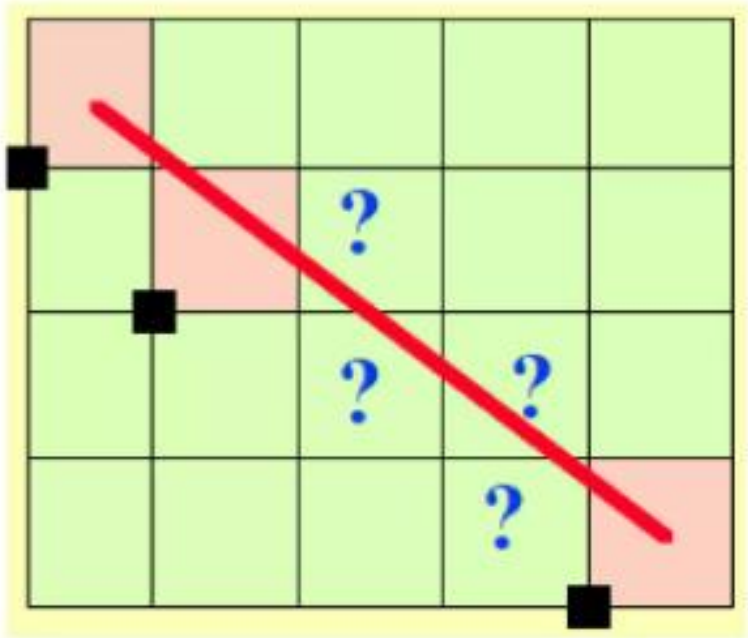


# Primitives

- A 2D drawing or a 3D object consists of graphical primitives such as points, lines, circles, and filled polygons.
- The graphics system or application program converts each primitive from its geometric definition into a set of pixels that represents the primitive in image space.
- This conversion is called scan conversion or rasterization.

# Definitions

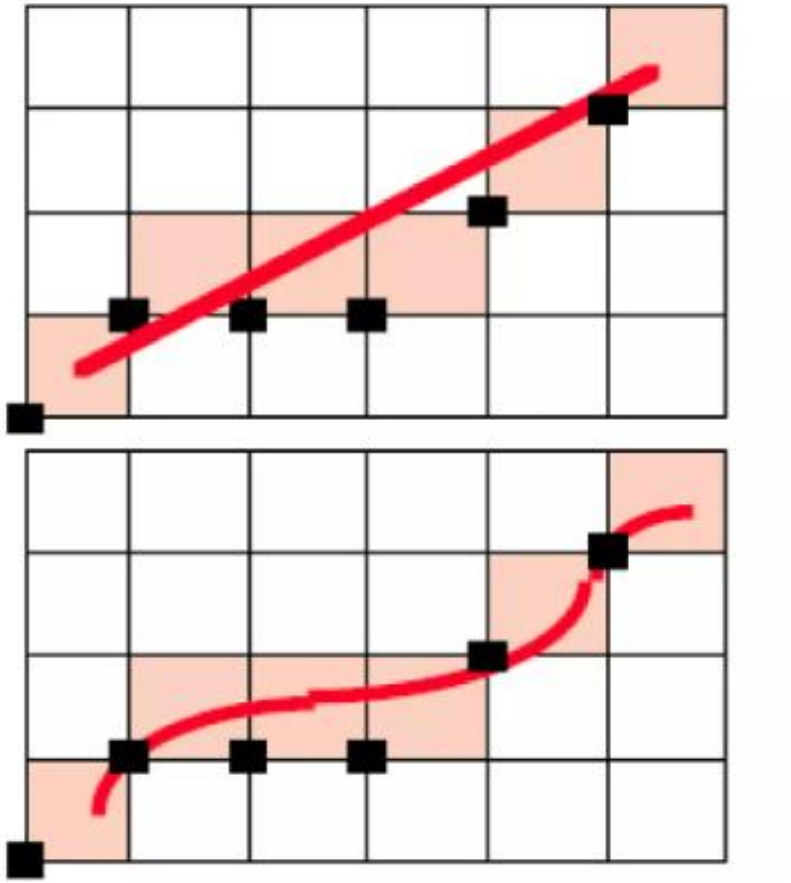
- **Rasterization** - The process of converting graphical primitives into pixels on the screen.
- **Scan conversion -** The process of selecting the nearest pixel positions to represent a geometric object on a raster display.



# General Requirements for Line Drawing

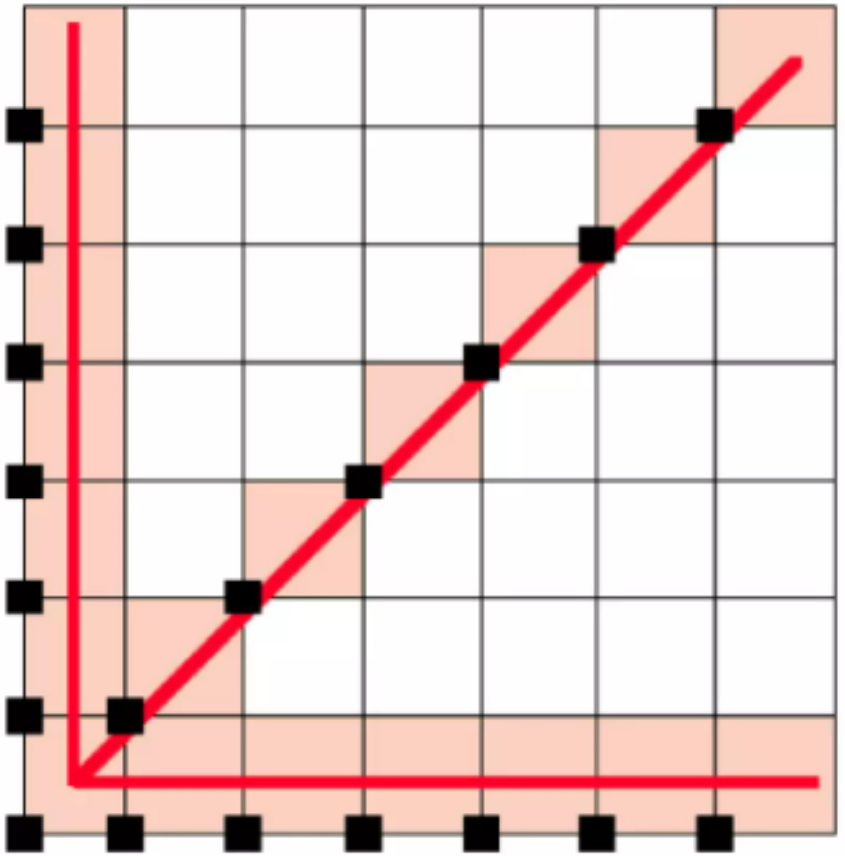
- Straight lines should appear visually straight.
- They should begin and end at the correct positions.
- Their brightness should remain uniform along the entire length.
- They should be generated efficiently and rapidly.

# Straight lines should appear visually straight



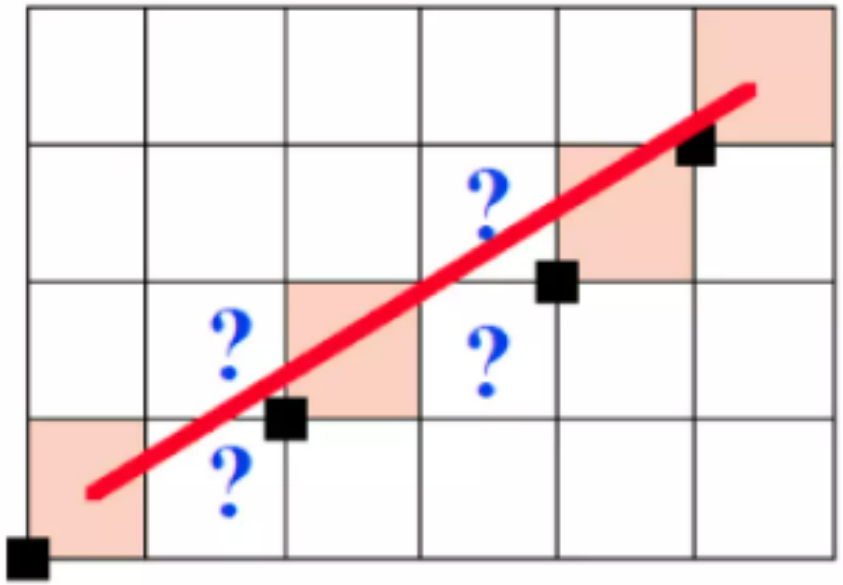
# horizontal, vertical, and 45° lines

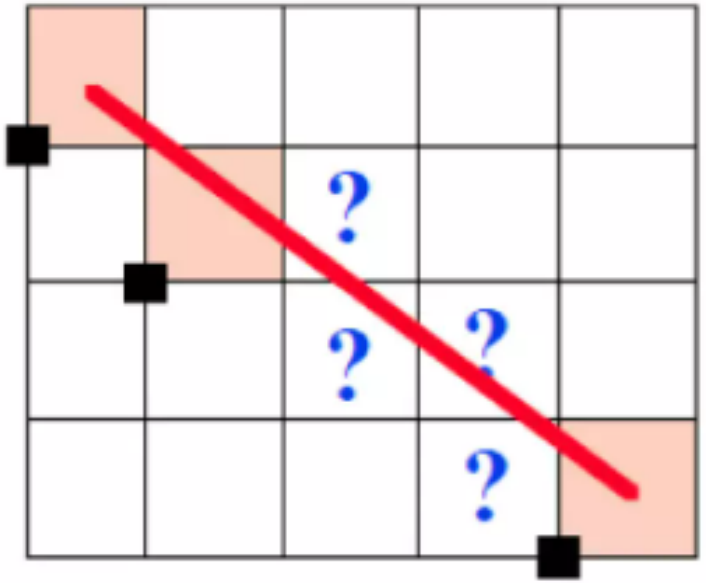
- For horizontal, vertical, and 45° lines, the choice of raster elements is straightforward. These lines exhibit constant brightness along their length.



# Lines with any other orientation

- For lines with any other orientation, choosing the appropriate pixels is more difficult.



# Intersection Points

- A mathematical line usually does not pass exactly through pixel intersection points.
- Because pixels are small and densely packed, the displayed line appears continuous.
- Therefore, we must choose the addressable pixels that best approximate the line.
- These pixels should be the ones closest to the true line.
- Pixel positions on the screen are discrete and use integer coordinates.

# Point Plotting

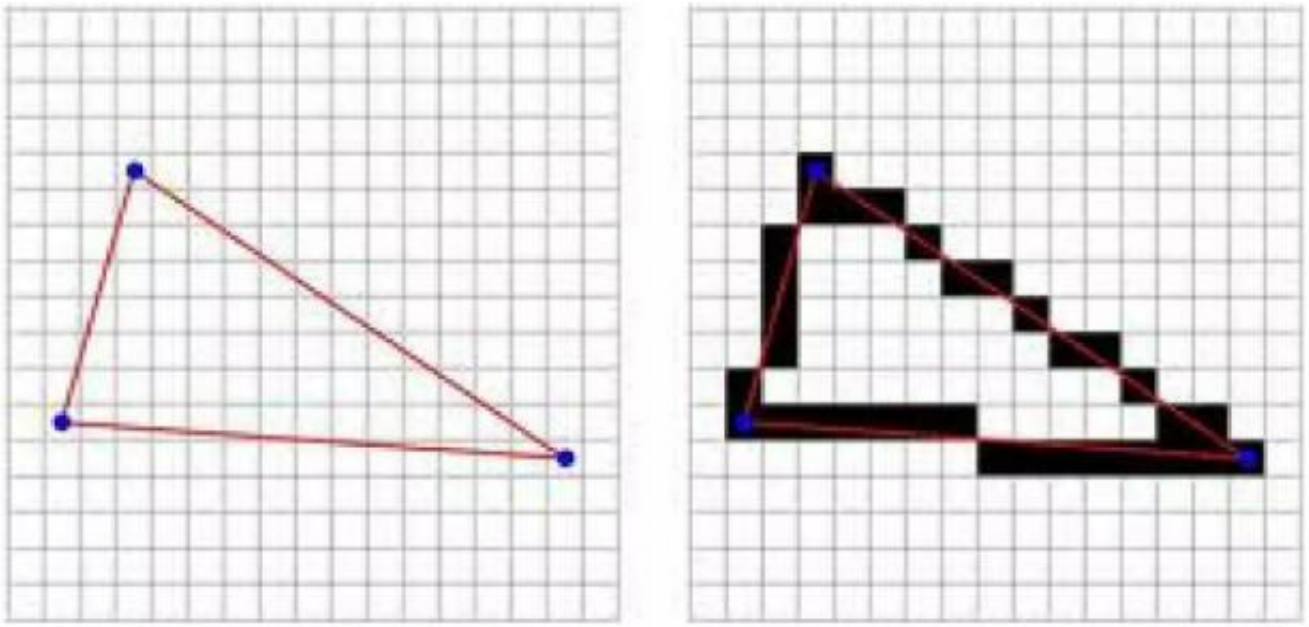
- A **point** is the simplest graphics primitive.
- A point is represented by a single coordinate: (x, y)

Example

- To draw a point at (5, 4), the system activates the pixel located at that coordinate.

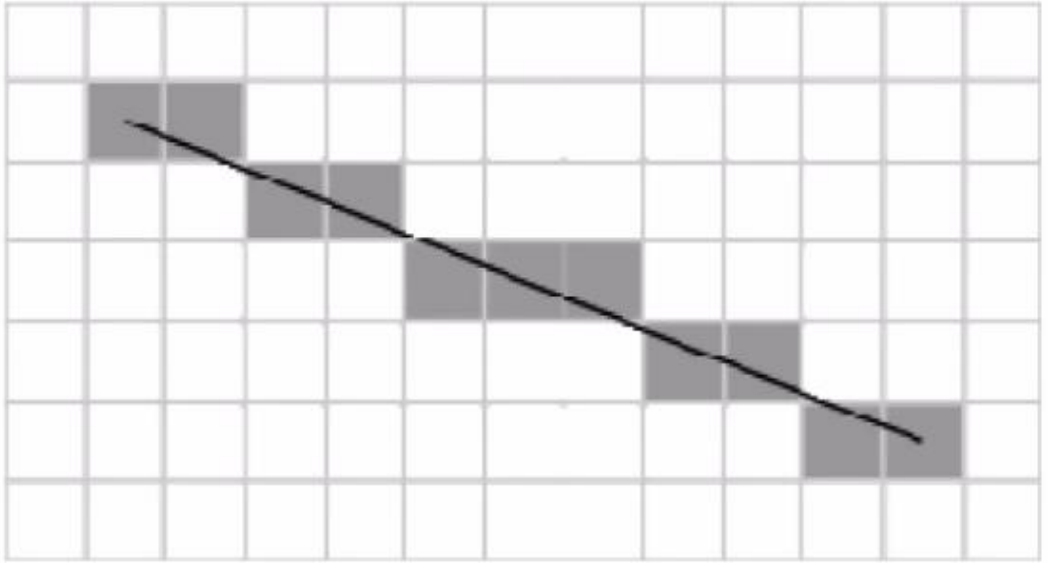
# Jaggies (Aliasing)

- When straight lines are drawn on a raster grid, they may appear jagged.
- This stair-step effect is known as jaggies or aliasing.
- It occurs because pixels are discrete square elements, while the actual line is continuous.



# Staircase Effect in Raster Lines

- A straight line on a raster display may appear as a sequence of small steps.
- This is called the staircase effect.
- It is more noticeable for diagonal lines and at low resolution.



# Line Drawing

- A line connects two points.
- Suppose a line starts at,
- (x1,y1)and ends at (x2,y2)
- A line is one of the most important primitives because many shapes are built using lines.

# Equation of a line

- From basic mathematics, a line can be written as $y = mx + c$
- Where
  - m= slope
  - c = intercept
- Slope is: $\frac{y_2 - y_1}{x_2 - x_1}$
- This formula is mathematically correct, but it is not always efficient for a computer because,
  - it may involve floating-point calculations
  - rounding errors may occur
  - we need fast algorithms for pixel plotting
- So special algorithms are used.

# DDA LINE ALGORITHM

- DDA (Digital Differential Analyzer) Line Algorithm
- DDA is a line drawing algorithm that calculates intermediate points between the starting and ending points of a line.
- Easy to understand
- Simple to implement
- Useful for drawing straight lines pixel by pixel

# Concept of DDA

- Suppose a line starts at: $(x1,y1)$
- and ends at: $(x2,y2)$
- Then: $dx = x2 - x1 \quad and \quad dy = y2 - y1$
- Number of steps: $steps = max(|dx|,|dy|)$
- Increment values: $x_{inc} = \frac{dx}{steps} \quad and \quad y_{inc} = \frac{dy}{steps}$

```
Algorithm DDA(x1, y1, x2, y2)
{
   dx = x2 - x1;
   dy = y2 - y1;

   if (abs(dx) > abs(dy))
      step = abs(dx);
   else
      step = abs(dy);

   xinc = dx / step;
   yinc = dy / step;

   x = x1;
   y = y1;

   putpixel(round(x), round(y));

   for (i = 1; i <= step; i++)
   {
      x = x + xinc;
      y = y + yinc;
      putpixel(round(x), round(y));
   }
}
```

# DDA Algorithm Steps

1. Input starting point $(x1, y1)$ and ending point $(x2, y2)$
2. Compute $dx = x2 - x1 \; and \; dy = y2 \; - y1$
3. Find number of steps:

   $$steps = \max([dx], [dy])$$
4. Compute increments:

   $$x_{inc} = \frac{dx}{steps} \quad y_{inc} = \frac{dy}{steps}$$
5. Initialize $x = x1 \; and \; y = y1$
6. Plot $\big(round(x), round(y)\big)$
7. Repeat for each step,

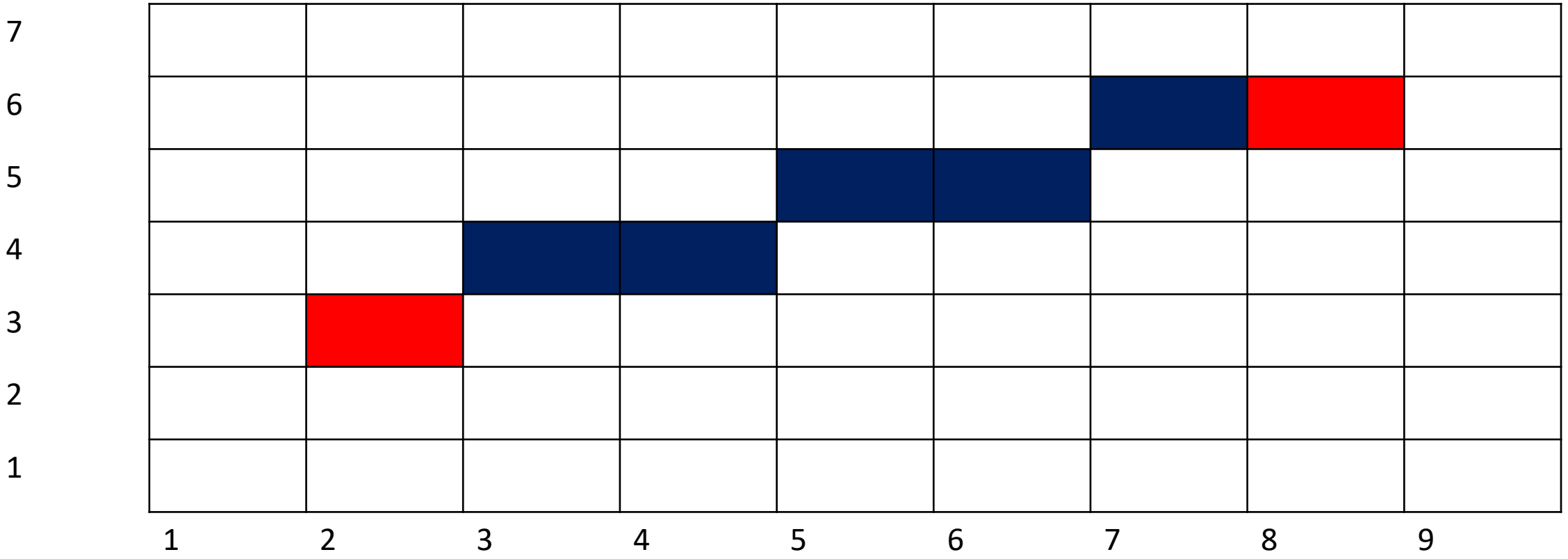
   $$x = x + x_{inc} \; , y = y + \; y_{inc}$$
8. Plot $\big(round(x), round(y)\big)$

# Example- Draw line from (2,3)to (8,6)

- $dx = 8 - 2 = 6\ and\ dy = 6 - 3 = 3$
- $steps = \max(6,3) = 6$
- $x_{inc} = \frac{6}{6} = 1\ y_{inc} = \frac{3}{6} = 0.5$

| Step | x | y | Plotted Pixel |
|---|---|---|---|
| 0 | 2.0 | 3.0 | (2, 3) |
| 1 | 3.0 | 3.5 | (3, 4) |
| 2 | 4.0 | 4.0 | (4, 4) |
| 3 | 5.0 | 4.5 | (5, 5) |
| 4 | 6.0 | 5.0 | (6, 5) |
| 5 | 7.0 | 5.5 | (7, 6) |
| 6 | 8.0 | 6.0 | (8, 6) |

# DDA Line Drawing



# Exercise

- Generate the pixel positions for the line from (4, 4) to (10, 8) using DDA.

# BRESENHAM LINE ALGORITHM

- Bresenham’s line algorithm is an efficient line drawing method that uses only integer calculations.
- **Why is it important?**
  - Faster than DDA
  - No floating point operations
  - More accurate for raster displays

```
Algorithm Bresenham(x1, y1, x2, y2)
{
   x = x1;
   y = y1;
   dx = x2 - x1;
   dy = y2 - y1;
   p = 2*dy - dx;
   while (x <= x2)
   {
      putpixel(x, y);
      x = x + 1;

      if (p < 0)
         p = p + 2*dy;
      else
      {
         p = p + 2*dy - 2*dx;
         y = y + 1;
      }
   }
}
```

# Basic Idea of Bresenham Line Algorithm

- When drawing a line, the next pixel can be one of two candidates.
- For a line with slope between 0 and 1:
  - Move $East(E) \rightarrow (x+1, y)$
  - Move $North\ East(NE) \rightarrow (x+1, y+1)$
- A **decision parameter** is used to decide which pixel is closer to the true line.

# Bresenham Line Formula

- For a line from $(x1, y1)\ to\ (x2, y2)$,
  - $dx = x2 - x1\ and\ dy = y2 - y1$
- Initial decision parameter $P_0 = 2dy - dx$
- Decision rules,
  - If $P_k < 0, choose\ E$
    - $P_{k+1} = P_k + 2dy$
  - If $P_k \geq 0, choose\ NE$
    - $P_{k+1} = P_k + 2dy - 2dx$

# Bresenham Line Algorithm Steps

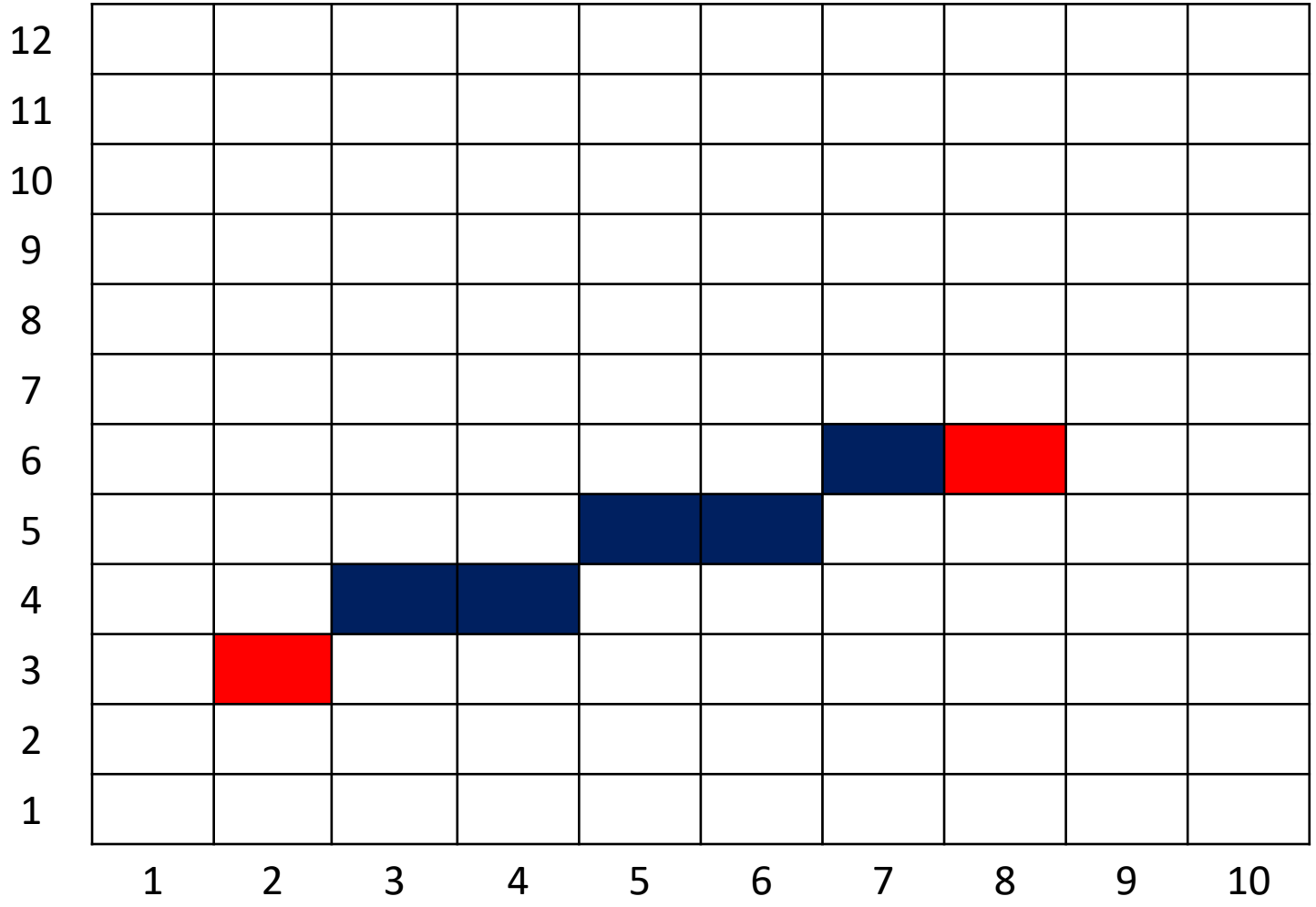
- Input $(x1, y1)$ and $(x2, y2)$
- Compute $dx = x2 - x1 \; and \; dy = y2 - y1$
- Initialize $\mathrm{p} = 2dy - dx$
- Start from $(x, y) = (x1, y1)$
- Plot the first point
- For each $x$ from $x1$ to $x2$:
  - If $p < 0$: $choose(x + 1, y)$
  - Else $choose(x + 1, y + 1)$
  - Update decision parameter
  - Plot the new point

# Example Draw line from (2,3)to (8,6)

- $dx = 6\ and\ dy = 3$
- $p_0 = 2(3) - 6 = 0$

| Step | Current Point | $(p_k)$ | Choice | Next Point | New (p) |
|---|---|---|---|---|---|
| 0 | (2,3) | 0 | $(p_k \geq 0)$ → NE | (3,4) | (0 + 6 - 12 = -6) |
| 1 | (3,4) | -6 | $(p_k < 0)$ → E | (4,4) | (-6 + 6 = 0) |
| 2 | (4,4) | 0 | $(p_k \geq 0)$ → NE | (5,5) | (0 + 6 - 12 = -6) |
| 3 | (5,5) | -6 | $(p_k < 0)$ → E | (6,5) | (-6 + 6 = 0) |
| 4 | (6,5) | 0 | $(p_k \geq 0)$ → NE | (7,6) | (0 + 6 - 12 = -6) |
| 5 | (7,6) | -6 | $(p_k < 0)$ → E | (8,6) | (-6 + 6 = 0) |

# Bresenham Line Drawing



# Exercise

- Apply Bresenham’s algorithm to determine the pixel positions for the line joining (1,2)and (7,5).

# DDA vs Bresenham Line Algorithm

| **Feature** | **DDA** | **Bresenham** |
|---|---|---|
| Arithmetic | Floating point | Integer |
| Speed | Slower | Faster |
| Accuracy | Less accurate | More accurate |
| Complexity | Simple | Slightly more complex |
| Rounding | Required | Minimal |

# Scan Converting Circles

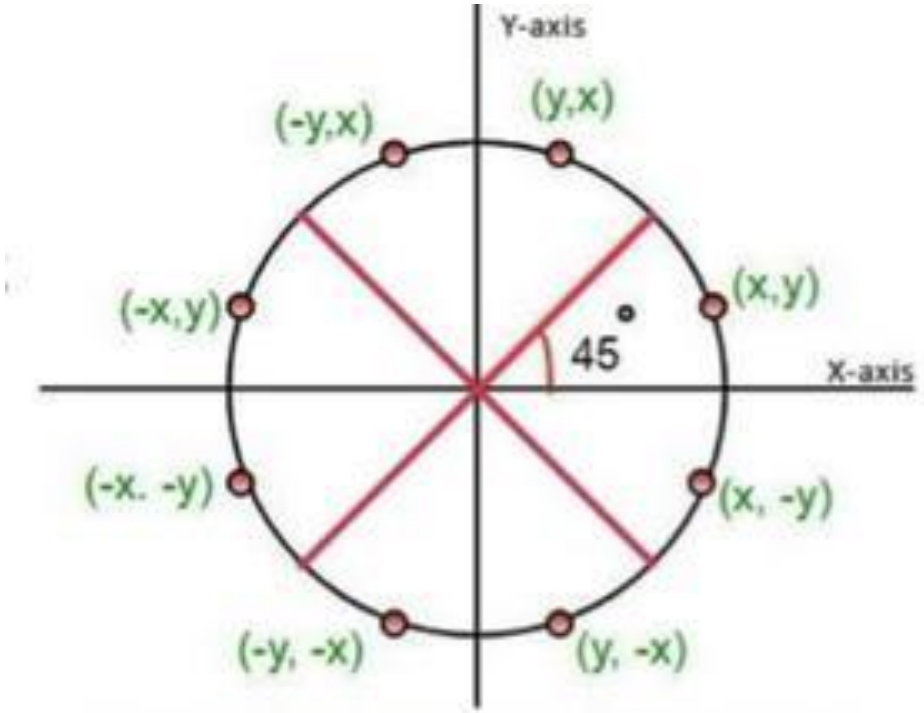
- A circle is defined as the set of points that are all at a given distance $\boldsymbol{r}$ from a center point $(x_c, y_c)$.
- For any circle point $(x, y)$ this distance is expressed by the equation:
  - $(x - x_c)^2 + (y - y_c)^2 = r^2$
- We can calculate points on the circle by stepping along the x-axis in unit steps from $x_c - r \; to \; x_c + r$ and then computing the corresponding y-values as:
  - $y = y_c \pm \sqrt{r^2 - (x - x_c)^2}$

# Problems with this approach

- If we generate circle points directly from $y = y_c \pm \sqrt{r^2 - (x - x_c)^2}$
- by moving along the x-axis one unit at a time, there are two main problems
- **High computation cost**
  At every step, we must evaluate a square root, which is computationally expensive.
- **Non-uniform pixel distribution**
  The plotted pixels are not evenly spaced around the circle.
  In some regions, pixels appear closer together, while in other regions they appear farther apart.

# Circle Symmetry

- Problem 1 can be reduced by exploiting the symmetry of the circle.
- If one point (x,y) on the circle is known, the other seven symmetric points can be obtained easily.
- Therefore, we only need to calculate points for one octant and reflect them to the other seven octants.



# Circle Symmetry

- The equation of a circle centered at origin is $x^2 + y^2 = r^2$
- A circle is symmetric in **8 octants**, so we only calculate points in one octant and reflect them.
- If $(x, y)$ is a point on the circle, then the following are also points,
  1. $(x, y)$
  2. $(y, x)$
  3. $(-x, y)$
  4. $(-y, x)$
  5. $(-x, -y)$
  6. $(-y, -x)$
  7. $(x, -y)$
  8. $(y, -x)$

# MIDPOINT CIRCLE ALGORITHM

- This algorithm is used to draw a circle by determining the nearest pixel position using a decision parameter.
- **Why use it?**
  - Efficient
  - Uses symmetry
  - Uses integer calculations

```
Algorithm MidpointCircle(xc, yc, r)
{
   x = 0;
   y = r;
   p = 1 - r;
   while (x <= y)
   {
      putpixel(xc + x, yc + y);
      putpixel(xc + y, yc + x);
      putpixel(xc - x, yc + y);
      putpixel(xc - y, yc + x);
      putpixel(xc - x, yc - y);
      putpixel(xc - y, yc - x);
      putpixel(xc + x, yc - y);
      putpixel(xc + y, yc - x);
      x++;
      if (p < 0)
         p = p + 2*x + 1;
      else
      {
         y--;
         p = p + 2*x + 1 - 2*y;
      }
   }
}
```

# Midpoint Circle Decision Parameter

- Start from $x = 0\ and\ y = r$
- Initial decision parameter: $p_0 = 1 - r$
- Decision rules
  - If $P < 0, choose\ East\ pixel$
    - $x_{k+1} = x_k + 1$
    - $y_{k+1} = y_k$
    - $p_{k+1} = p_k + 2x + 3$
  - If $P \geq 0, choose\ South\ East\ pixel$
    - $x_{k+1} = x_k + 1$
    - $y_{k+1} = y_k - 1$
    - $p_{k+1} = p_k + 2x - 2y + 5$
  - Continue until
    - $x \geq y$

# Midpoint Circle Algorithm Steps

1. Input radius $r\ and\ center(x_c, y_c)$
2. Initialize $x = 0, y = r, p = 1 - r$
3. Plot the 8 symmetric points
4. While $x < y$,
   - Increment $x$
   - If $p < 0$, update: $p = p + 2x + 3$
   - Else
     - Decrement $y$
     - Update: $p = p + 2x - 2y + 5$
   - Plot the 8 symmetric points

# Example Draw a circle with radius r=5

- Initial values - $x = 0,\ y = 5,\ p = 1 - 5 = -4$

| Step | $x_k$ | $y_k$ | $p_k$ | Condition | Choice | $x_{k+1}$ | $y_{k+1}$ | $p_{k+1}$ |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 5 | -4 | $(p_k < 0)$ | E | 1 | 5 | (p_1 = -4 + 2(0) + 3 = -1) |
| 1 | 1 | 5 | -1 | $(p_k < 0)$ | E | 2 | 5 | (p_2 = -1 + 2(1) + 3 = 4) |
| 2 | 2 | 5 | 4 | $(p_k \geq 0)$ | SE | 3 | 4 | (p_3 = 4 + 2(2) - 2(5) + 5 = 3) |
| 3 | 3 | 4 | 3 | $(p_k \geq 0)$ | SE | 4 | 3 | (p_4 = 3 + 2(3) - 2(4) + 5 = 6) |

# Example Draw a circle with radius r=5

| Symmetric Points | Symmetric Points | Symmetric Points | Symmetric Points |
|---|---|---|---|
| (0,5) | (1,5) | (2,5) | (3,4) |
| (5,0) | (5,1) | (5,2) | (4,3) |
| (0,-5) | (-1,5) | (-2,5) | (-3,4) |
| (-5,0) | (-5,1) | (-5,2) | (-4,3) |
| | (-1,-5) | (-2,-5) | (-3,-4) |
| | (-5,-1) | (-5,-2) | (-4,-3) |
| | (1,-5) | (2,-5) | (3,-4) |
| | (5,-1) | (5,-2) | (4,-3) |

# Example Draw a circle with radius r=5

| Step | Point in First Octant | Symmetric Points Generated |
|---|---|---|
| 0 | (0,5) | (0,5), (5,0), (0,-5), (-5,0) |
| 1 | (1,5) | (1,5), (5,1), (-1,5), (-5,1), (-1,-5), (-5,-1), (1,-5), (5,-1) |
| 2 | (2,5) | (2,5), (5,2), (-2,5), (-5,2), (-2,-5), (-5,-2), (2,-5), (5,-2) |
| 3 | (3,4) | (3,4), (4,3), (-3,4), (-4,3), (-3,-4), (-4,-3), (3,-4), (4,-3) |

# Exercise

- Determine the first-octant points for a circle of radius 7 using the Midpoint Circle Algorithm.

# Advantages of Midpoint Circle Algorithm

- **Advantages**
  - Efficient and fast
  - Uses integer arithmetic
  - Reduces calculations using 8-way symmetry
  - Suitable for raster displays
- **Applications**
  - Drawing circles in paint systems
  - Games and animations
  - Graphical user interfaces
  - CAD systems

# BRESENHAM CIRCLE ALGORITHM

- Bresenham circle algorithm is an efficient method to draw circles using integer calculations.

```
Algorithm BresCircle(xc, yc, r)
{
   x = 0;
   y = r;
   d = 3 - 2*r;
   while (x <= y)
   {
      putpixel(xc + x, yc + y);
      putpixel(xc - x, yc + y);
      putpixel(xc + x, yc - y);
      putpixel(xc - x, yc - y);
      putpixel(xc + y, yc + x);
      putpixel(xc - y, yc + x);
      putpixel(xc + y, yc - x);
      putpixel(xc - y, yc - x);
      if (d < 0)
         d = d + 4*x + 6;
      else
      {
         d = d + 4*(x - y) + 10;
         y--;
      }
      x++;
   }
}
```

# Working Principle of Bresenham Circle Algorithm

- Start with $x = 0, y = r$
- Decision parameter is initialized and updated at each step.
- At each iteration, choose between:
  - East pixel
  - South-East pixel
- Use 8-way symmetry to draw the full circle.

# Bresenham Circle Algorithm Formula

- Initial decision parameter can be written as $d = 3 - 2r$
- Decision rules:
  - If $d < 0$:
    - $d = d + 4x + 6$
  - Else
    - $d = d + 4(x - y) + 10$

    and decrement $y$
  - At each step, increment $x$.
- Stop when $x > y$

# Bresenham Circle Algorithm Steps

1. Input radius $r\ and\ center(x_c, y_c)$
2. Initialize $x = 0,\ y = r,\ d = 3 - 2r$
3. Plot the 8 symmetric points
4. While $x \leq y$:
   - Plot points
   - If $d < 0$:
     - $d = d + 4x + 6$
   - Else:
     - $d = d + 4(x - y) + 10$
     - $y = y - 1$
   - Increment $x = x + 1$

# **Example:** Draw circle with radius r=5

- Initial values $x = 0, y = 5, d = 3 - 2(5) = -7$

| Step | x | y | d | Decision |
|---|---|---|---|---|
| 0 | 0 | 5 | -7 | Start |
| 1 | 1 | 5 | -1 | Choose E |
| 2 | 2 | 5 | 9 | Choose E |
| 3 | 3 | 4 | 7 | Choose SE |
| 4 | 4 | 3 | stop | Since x > y |

- Then draw the 8 symmetric points for each $(x, y)$

# Exercise

- circle with radius r=7 and determine all the symmetric points indicating all necessary calculations